Engine & Game Design

Contents

Project Links 2

Engine Design 2

Engine Architecture UML Diagram 4

System Details 4

Input System & Input Handler 4

Movement System & Collider Sync System 5

Projectile Spawn System 5

Physics System 5

Collision System & Collision Handler 5

Post-Collision Systems 5

Shooting UI Systems 6

Camera System 6

Rendering Systems 6

Animation Systems 6

Audio System 7

Turn Management System 7

Game State System & Game Over Handler 7

Game Design 7

Game Features 7

Game Control 8

Game Narrative 8

Game Art Assets 8

Tools and Risks 9

Tools 9

Risks 9

Roles	Owner
• Project owner • Engine designer • Engine developer • Gameplay designer • Gameplay developer • Documentation	Kim Chung

Project Links

GitHub: https://github.com/Kimmm-c/Stone.git

Trello:
https://trello.com/invite/b/69a908abc5184e7465a1f5dc/ATTI8909eacc573b8b5fafd8c9a3fad3f7d7DC8D9256/stone-engine

Engine Design

The proposed engine is a 2D projectile-based game framework designed to support turn-based local multiplayer games where players launch projectiles affected by physics, similar to artillery-style games such as Angry Birds or Worms. The engine will be designed in a modular way so that it can support other games within the same genre. The framework will manage gameplay features such as projectile motion, destructible terrain, player health, turn-based gameplay, camera tracking, and collision interaction between players, terrain, and projectiles.

Figure 1. Worms (1995)

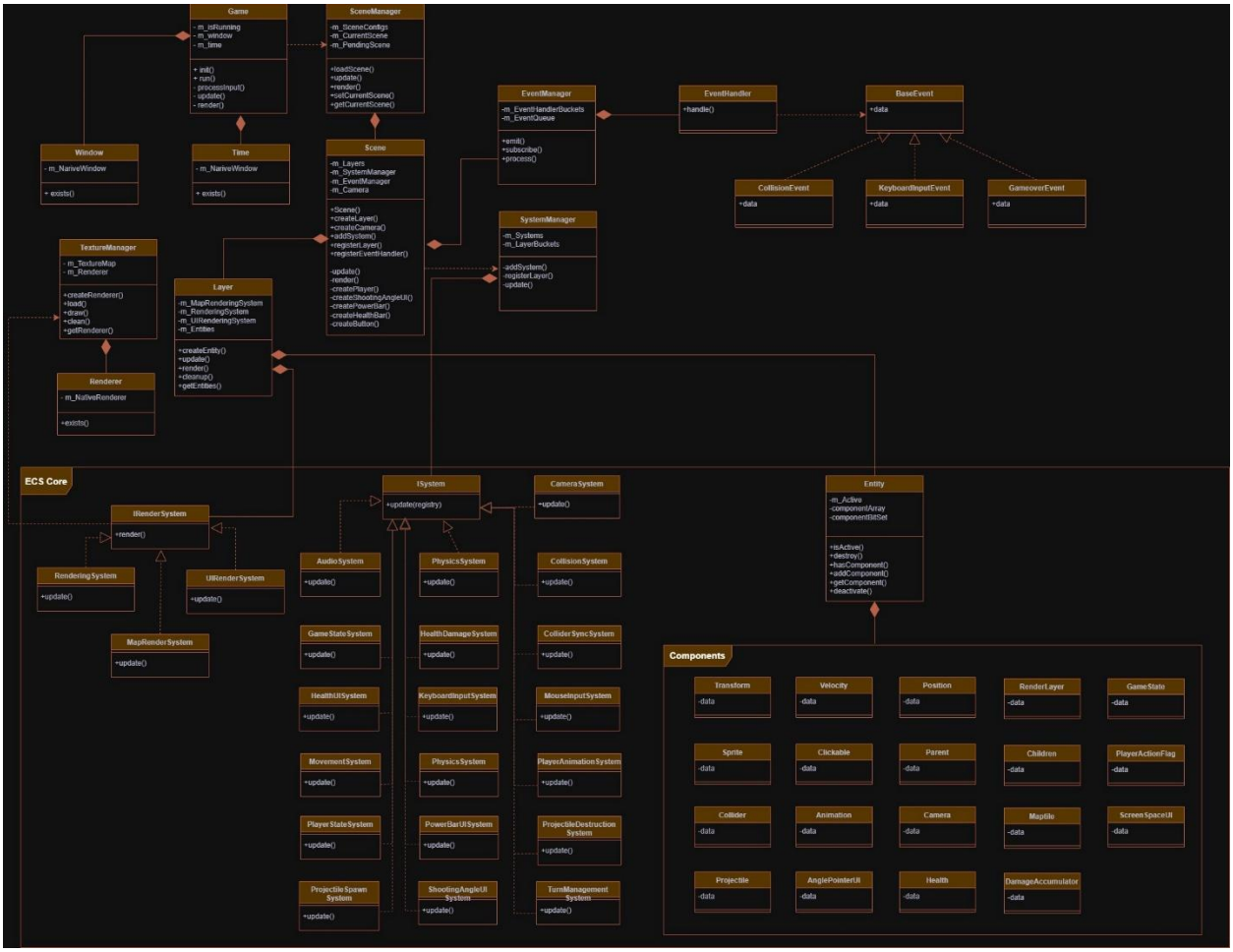
The engine will be based on the architecture and concepts introduced in the course lectures, including the use of an ECS design pattern. The lecture material will serve as the foundation for the engine structure, and it will be modified where necessary to support gameplay mechanics such as destructive terrain and projectile spawning.

The project will be implemented from scratch using the SDL multimedia library. Other than window creation, rendering, and timing, SDL is also used to handle keyboard input for player controls such as adjusting shooting angle, charging shooting power, selecting shooting type, and activating teleport abilities.

The engine will follow a pure ECS architecture similar to the one introduced in the lectures. In this design, entities represent game objects, components store data such as position, velocity, health, and collision information, and systems contain the logic that operates on entities with specific component combinations. Core systems in the engine will include an input system, physics system, collision system, rendering system, camera system, projectile system, terrain destruction system, and turn management system.

Finally, the engine will implement a central game loop responsible for processing input, updating systems, and rendering the scene each frame. The camera system will dynamically follow active projectiles during flight and return to the active player during aiming phases. By structuring the engine around modular ECS systems, the framework will provide a flexible foundation for developing other 2D projectile mechanics games beyond the initial project.

Engine Architecture UML Diagram



System Details

Input System & Input Handler

The Input System reads keyboard events provided by SDL and determines how the active player interacts with the game. It emits input events that are processed by input handlers.

These handlers update player movement direction, adjust shooting angle and shooting force, and assign a PendingProjectileTag when a shot is triggered. This tag signals the Projectile Spawn System to create a projectile entity.

Only the active player is allowed to provide input during their turn.

Movement System & Collider Sync System

The Movement System updates entity positions based on their velocity. It is responsible for moving players during their turn.

After movement updates, the Collider Sync System ensures that collider components remain aligned with the updated entity positions. This keeps collision detection accurate.

Projectile Spawn System

The Projectile Spawn System is responsible for creating projectile entities. It searches for entities with a PendingProjectileTag and generates projectiles accordingly.

The system calculates initial projectile velocity using trigonometry based on shooting angle and force.

Physics System

The Physics System is responsible for updating the movement of entities affected by physical forces. This includes projectiles and players that are influenced by gravity.

The system calculates changes in position and velocity based on gravitational acceleration and other forces. A projectile's trajectory is determined by the initial shooting angle and power applied during launch. Players are also subject to gravity and will fall if the terrain beneath them is destroyed. This system ensures that the game world behaves realistically.

Collision System & Collision Handler

The Collision System detects interactions between entities in the game world. It determines when two objects intersect and generates appropriate collision events.

Some collision types required for the gameplay mechanics are:

- **Player vs terrain:** Stops player movement
- **Player vs projectile**: Tracks overlap using a DamageAccumulator component
- **Projectile vs terrain** Assigns PendingDestroy tag

The DamageAccumulator stores the maximum overlap between player and projectile, which is later used to calculate damage.

Post-Collision Systems

After collision events are resolved, several systems process the results:

- **HealthDamage System**: Converts overlap values into damage and reduces player health

- **HealthUISync System**: Updates the health bar UI
- **ProjectileDestruction System**: Removes projectiles marked with PendingDestroy

These systems ensure gameplay consequences are applied correctly after collisions.

Shooting UI Systems

- **ShootingAngleUISync System:** Updates the shooting angle indicator based on the player's input and facing direction, ensuring that the UI reflects the current aiming direction.
- **PowerbarUISync System:** Maps the current shooting force to a visual power bar, providing real-time feedback to the player as they charge their shot. These systems help bridge gameplay mechanics with visual feedback.

Camera System

The Camera System manages the visible portion of the game world and determines which object the camera should follow at any given time. During the aiming phase, the camera focuses on the active player. After a projectile is launched, the camera transitions to follow the moving object until it lands or collides with another object. Once the projectile interaction finishes, the camera returns to the next active player.

Rendering Systems

Rendering responsibilities are divided across three systems to maintain clear separation of concerns.

- **MapRender System**: Responsible for drawing terrain and background elements.
- **GameplayRender System**: Renders dynamic entities such as players and projectiles.
- **UIRender System**: Handles all user interface elements, including health bars and shooting indicators. This layered rendering approach ensures proper visual organization and clarity.

Animation Systems

Animation is handled through a set of coordinated systems.

- **PlayerState System**: Determines the current state of the player, such as idle or falling.
- **PlayerAnimation System**: Selects the appropriate animation clip based on that state.
- **AnimationPlayback System**: Responsible for playing and updating animations over time. Together, these systems provide responsive visual feedback tied to gameplay actions.

Audio System

The Audio System manages sound playback using SDL3_Mixer. It is responsible for playing sound effects such as projectile launches, collisions, and UI interactions, enhancing the overall gameplay experience.

Turn Management System

This system controls the flow of the game by alternating turns between the two players. At the beginning of a turn, this system assigns the active player and enables their input controls.

Once a projectile has been launched, the system waits until the action has fully resolved. This includes projectile movement, collisions, terrain destruction, and player damage. After the projectile interaction is complete and the game state becomes stable, the system switches control to the other player. This system ensures that only one player can act at a time.

Game State System & Game Over Handler

This system monitors overall game conditions and determines when the match ends. It checks whether a player's health has reached zero or if a player has fallen out of the playable area due to terrain destruction.

When one of these conditions occurs, the system declares the opposing player as the winner and emits an event. The Game Over Handler responds to this event by displaying an end-game overlay to the player.

Game Design

Game Features

Stone Duel is a local two-player, turn-based 2D artillery game where players take turns launching stones at each other using adjustable angles and shooting power. The game takes place on a destructible terrain where every projectile impact alters the environment, forcing players to continuously adapt their positioning and strategy. Players must carefully aim their shots while accounting for gravity and terrain deformation in order to eliminate their opponent.

The game is designed to run on desktop platforms (Windows) and will support local multiplayer on a shared screen. Both players play on the same computer (using different input sets) and take turns controlling their character.

The core gameplay revolves around projectile-based combat. During their turn, a player adjusts the shooting angle of their stone and charges shooting power by holding a key. When the key is released, the stone is launched as a projectile affected by gravity.

When a projectile collides with a player, the player loses health. At the same time, the terrain where the projectile lands is damaged, creating a crater in the ground. Because the terrain is destructible, players may lose the ground beneath them and fall due to gravity.

The game ends when one player loses all their health or falls off the map due to terrain destruction.

Game Control

The game supports local two-player gameplay on a shared keyboard. Each player uses a different set of keys to control their character during their turn. Only the active player can provide input.

Action	Player 1 (Left side of the screen)	Player 2 (Right side of the screen)
Adjust shooting angle	W/S	Up/Down arrow keys
Charge shot power	Hold E	Hold /
Move horizontally	A/D	Left/Right arrow keys

Game Narrative

The game features a simple narrative centered around two rival stone throwers competing in a duel across a destructible battlefield. The characters are positioned on uneven terrain and must eliminate one another using careful aiming and environmental destruction.

Game Art Assets

The game uses a combination of custom-created assets and free online resources to balance development efficiency and visual quality.

UI elements will be digitally designed and created by the developer. These include components such as the health bar, power charge bar, shooting angle indicator, and interactive buttons.

Character visuals, character animations, and 2D background assets will be sourced from free online asset libraries.

Tools and Risks

Tools

- **Game development:** The primary framework will be SDL. SDL will provide core functionality such as window management, input handling, and audio playback. Additional SDL modules or third-party utilities may also be incorporated if necessary.
- **Level design:** Tiled will be used to create tile-based maps and define collision layers.
- **Visual assets:** Procreate is used to create 2D artwork.

Risks

- **Time constraints:** The primary risk associated with this project is time constraints. The project must be completed in six weeks, which includes developing the game engine, implementing gameplay systems, designing levels, and creating art assets.
- **Art asset production:** Since artwork will likely be created manually using Procreate, the time required to produce assets may compete with development time. If asset production or development takes longer than expected, placeholder graphics may need to be used to prioritize engine and gameplay implementation.
- **Technical familiarity risk:** Additional experimentation with SDL subsystems will likely be required. This may introduce delays while learning how to properly integrate these systems into the engine.

To mitigate these risks, the project will prioritize implementing a minimal but functional engine architecture early in development. If necessary, the visual assets may be reduced to ensure the game engine and core game remain stable and complete within the available timeframe.